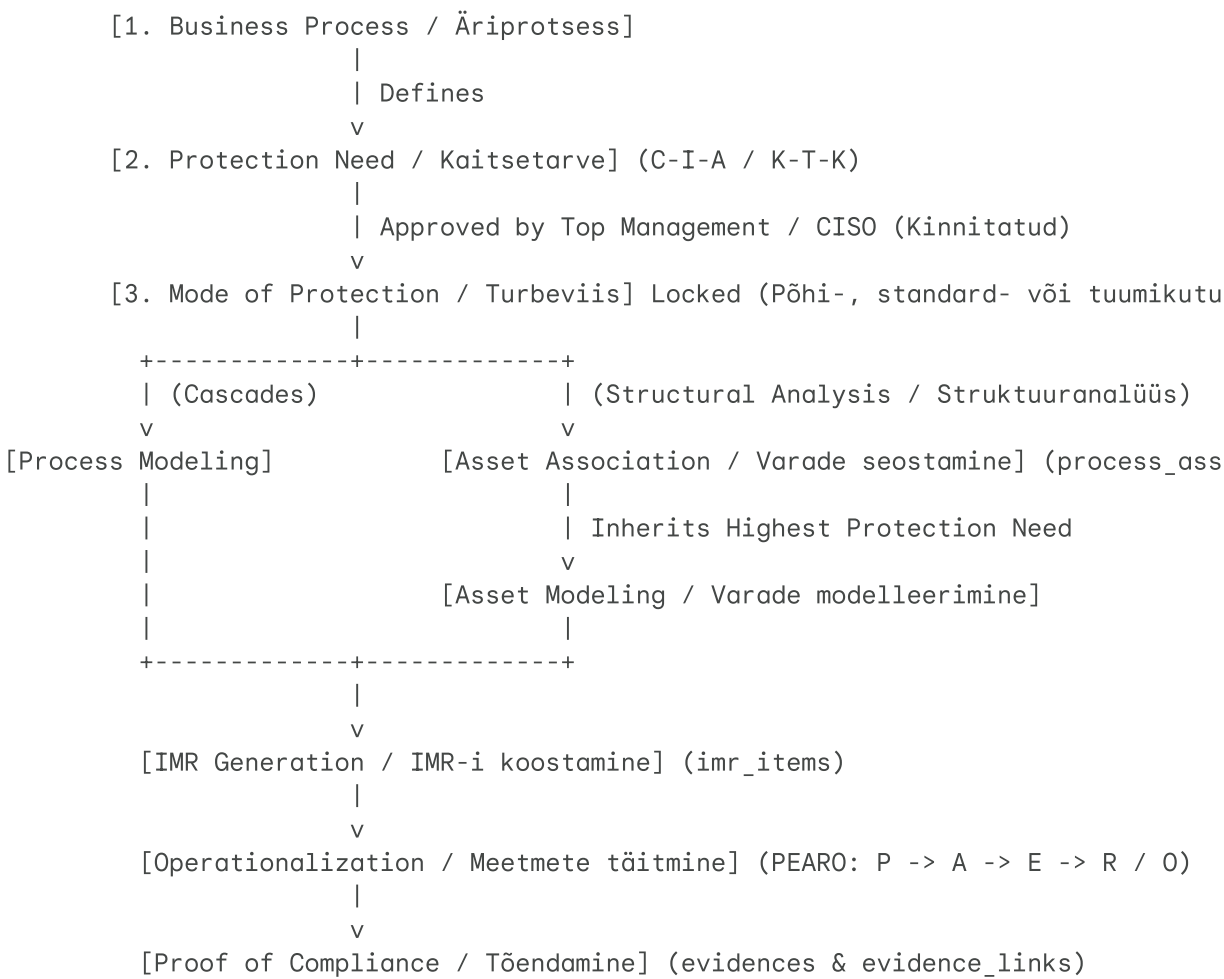


TECHNICAL DESIGN DOCUMENT: E-ITS v2 IMR & Scope Modeling System Architecture

This architectural blueprint outlines the design, implementation, and database relationships of the E-ITS (Estonian Information Security Standard - *Eesti infoturbestandard*) v2 system. It covers **Scope Modeling (*Kaitseala modelleerimine*)** and the **Infosec Measures Implementation Plan (IMR - *Infoturbe meetmete rakenduskava*)**, fully compatible with the database schema defined in `20260521181936_schema.sql`.

1. System Domain Concept & Workflow

E-ITS is a **business-process-centric standard (*äriprotsessikeskne standard*)**. Security starts at the business level and cascades to technical systems.



Core Business Rules & Validations:

- The Lockout Rule (*Lukustusreegel*):** Once the Tenant's Mode of Protection (*turbeviis*) is saved, the Protection Need (*kaitsetarve*) of any associated Business Process (*äriprotsess*) becomes **LOCKED** (cannot be edited). This guarantees that changes to risk levels do not silently invalidate already modeled baseline controls.

(*äriprotsess*) can have security modules modeled unless its Protection Need (*kaitsetarve*) is fully evaluated and marked as **Approved (*kinnitatud*)** by the CISO or Management.

3. **PEARO Status Safeguard (*PEARO-oleku kontroll*)**: A measure cannot transition to the **Implemented (*R - Rakendatud*)** status unless:
 - `implementation_details` (*rakendamise kirjeldus*) contains a comprehensive explanation (min 15 characters).
 - At least one active Evidence (*asitõend*) file metadata record is linked to the IMR item via the polymorphic `evidence_links` table.

2. Database Schema Alignment

This architecture matches the physical tables present in `20260521181936_schema.sql`.

Key Entities & Relations Mapping:

- `tenants` : Houses the tenant baseline configuration. We store `mode_of_protection` (`BASELINE` , `STANDARD` , or `CORE`) here.
- `business_processes` : Contains processes. Protection levels are mapped via `protection_need_summaries` (which references `business_processes.id`).
- `assets` : Houses target objects (*sihtobjektid*).
- `process_assets` : The join table implementing the Structural Analysis (*struktuuranalüüs*), linking processes to assets.
- `imr_items` : The core execution table. References:
 - `tenant_id` -> `tenants(id)`
 - `measure_id` -> `eits_catalog_measures(id)`
 - `responsible_id` -> `local_users(id)` (points to the assigned officer / *vastutaja*)
 - `created_by` / `updated_by` -> `local_users(id)`
- `evidences` & `evidence_links` : The polymorphic evidence management system. `evidence_links` references `imr_items.id` as `target_id` when `target_type` = `'imr_item'`.

SQL Setup for Custom Scope Modeling Mapping Table:

To assign modules (`eits_catalog_modules`) to processes or assets and auto-generate IMR rows, create the `mapped_modules` table:

```
CREATE TABLE public.mapped_modules (
  id uuid DEFAULT gen_random_uuid() NOT NULL,
  tenant_id uuid NOT NULL,
  module_id uuid NOT NULL,
  target_type character varying(50) NOT NULL,
  target_id uuid NOT NULL,
  created_at timestamp with time zone DEFAULT now(),
  created_by uuid,
  CONSTRAINT pk_mapped_modules PRIMARY KEY (id),
  CONSTRAINT fk_mapped_mod_tenant FOREIGN KEY (tenant_id) REFERENCES public.tenants(id),
  CONSTRAINT fk_mapped_mod_catalog FOREIGN KEY (module_id) REFERENCES public.eits_catalog_modules(id),
  CONSTRAINT fk_mapped_mod_creator FOREIGN KEY (created_by) REFERENCES public.local_users(id),
  CONSTRAINT uq_tenant_module_target UNIQUE (tenant_id, module_id, target_type),
);
```

```
-- index for speedy lookup during mapping & deletion
CREATE INDEX ix_mapped_modules_lookup ON public.mapped_modules USING btree (target_id)

-- Attach mapped_module_id as metadata on imr_items to track which mapping generated
ALTER TABLE public.imr_items
ADD COLUMN IF NOT EXISTS mapped_module_id uuid,
ADD CONSTRAINT fk_imr_mapped_module FOREIGN KEY (mapped_module_id) REFERENCES public.mapped_modules (id)
```

3. Backend (v2 API) Architecture & Service Logic

All controllers, validators, and entities must reside strictly under `backend/app/api/v2/`.

A. Modeling Service (`backend/app/services/v2/modeling_service.py`)

This service validates structural preparedness, locks process changes, maps modules, and generates the required IMR baseline lines based on the tenant's Chosen Mode of Protection (*turbeviis*).

```
import uuid
from typing import Dict, Any, List
from sqlalchemy.orm import Session
from fastapi import HTTPException, status
from app.models import (
    Tenant, Asset, BusinessProcess, MappedModule, ImrItem,
    EitsCatalogMeasure, ProcessAsset, ProtectionNeedSummary
)

class ModelingService:
    @staticmethod
    def validate_target_ready(db: Session, tenant_id: uuid.UUID, target_type: str) -> bool:
        """
        E-ITS Business Rule: Verify that target is approved for security modeling
        """
        if target_type == "business_process":
            # Verify the business process has an approved protection need
            summary = db.query(ProtectionNeedSummary).filter(
                ProtectionNeedSummary.business_process_id == target_id,
                ProtectionNeedSummary.tenant_id == tenant_id
            ).first()
            if not summary or not summary.approved_by:
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail="Tõrge: Äriprotsessi ei saa modelleerida, sest selle kaitsetaseme kiitust ei ole."
                )
        elif target_type == "asset":
            # Structural Analysis Check: Must be linked to at least one process
            relations = db.query(ProcessAsset).filter(
                ProcessAsset.asset_id == target_id,
                ProcessAsset.tenant_id == tenant_id
            ).all()
            if not relations:
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail="Tõrge: Seda vara ei saa modelleerida, sest see on seotud ühegi protsessiga."
                )

        # Verify that all linked processes are approved
        if target_type == "business_process":
            processes = db.query(BusinessProcess).filter(
                BusinessProcess.id == target_id,
                BusinessProcess.tenant_id == tenant_id
            ).all()
            for process in processes:
                if not process.approved_by:
                    raise HTTPException(
                        status_code=status.HTTP_400_BAD_REQUEST,
                        detail="Tõrge: Seda protsessi ei saa modelleerida, sest see ei ole kiidetud."
                    )
```

```

summary = db.query(ProtectionNeedSummary).filter(
    ProtectionNeedSummary.business_process_id == rel.business_pro
    ProtectionNeedSummary.tenant_id == tenant_id
).first()
if not summary or not summary.approved_by:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=f"Tõrge: Seotud äriprotsessi '{rel.business_proces
    )
return True

@staticmethod
def map_module(
    db: Session,
    tenant_id: uuid.UUID,
    user_id: uuid.UUID,
    module_id: uuid.UUID,
    target_type: str,
    target_id: uuid.UUID
) -> Dict[str, Any]:
    """
    Performs baseline modeling mapping and auto-populates imr_items matching
    """

    # Validate target preparedness
    ModelingService.validate_target_ready(db, tenant_id, target_type, target_

    # Check for pre-existing mapping
    existing = db.query(MappedModule).filter(
        MappedModule.tenant_id == tenant_id,
        MappedModule.module_id == module_id,
        MappedModule.target_type == target_type,
        MappedModule.target_id == target_id
    ).first()
    if existing:
        raise HTTPException(status_code=400, detail="See moodul on sellele ob

    # Create the mapping
    mapping = MappedModule(
        tenant_id=tenant_id,
        module_id=module_id,
        target_type=target_type,
        target_id=target_id,
        created_by=user_id
    )
    db.add(mapping)
    db.flush()

    # Query Tenant Security Profile
    tenant = db.query(Tenant).filter(Tenant.id == tenant_id).first()
    mode = tenant.mode_of_protection if tenant else "BASELINE"

    # Fetch catalog measures
    catalog_measures = db.query(EitsCatalogMeasure).filter(EitsCatalogMeasure
    generated_count = 0

    for measure in catalog_measures:
        # Filter according to mode of protection
        # BASELINE -> only BASE measures
        # STANDARD/CORE -> BASE and STANDARD measures
        should_apply = False
        if mode == "BASELINE" and measure.level == "BASE":

```

```

    elif mode in ["STANDARD", "CORE"] and measure.level in ["BASE", "STANDARD"]:
        should_apply = True

    if should_apply:
        # Build unique IMR item if not duplicate
        existing_imr = db.query(ImrItem).filter(
            ImrItem.tenant_id == tenant_id,
            ImrItem.measure_id == measure.id,
            ImrItem.asset_id == (target_id if target_type == "asset" else
                                ImrItem.business_process_id == (target_id if target_type == "business" else None)),
        ).first()

        if not existing_imr:
            imr_item = ImrItem(
                tenant_id=tenant_id,
                measure_id=measure.id,
                mapped_module_id=mapping.id,
                asset_id=target_id if target_type == "asset" else None,
                business_process_id=target_id if target_type == "business" else None,
                status="P", # Default to Planned (Kavandatud)
                priority="P2",
                created_by=user_id,
                updated_by=user_id
            )
            db.add(imr_item)
            generated_count += 1

    db.commit()
    return {
        "message": "Moodul edukalt modelleeritud ja IMR-i kirjed genereeritud",
        "mapped_module_id": mapping.id,
        "generated_measures_count": generated_count
    }

```

B. IMR Compilation Service (backend/app/services/v2/imr_service.py)

This service manages transitions through the PEARO statuses, locking protection need modifications when a mode of protection is active, and handling S3 evidence files.

```

import uuid
import datetime
from typing import Dict, Any
from sqlalchemy.orm import Session
from fastapi import HTTPException, status
from app.models import ImrItem, EvidenceLink, LocalUser, Tenant, ProtectionNeedStatus

class ImrService:
    @staticmethod
    def update_imr_item(
        db: Session,
        tenant_id: uuid.UUID,
        user_id: uuid.UUID,
        imr_item_id: uuid.UUID,
        update_data: Dict[str, Any]
    ) -> Dict[str, Any]:
        """
        Updates an IMR item's implementation parameters. Enforces S3 evidence att

```

```

item = db.query(ImrItem).filter(ImrItem.id == imr_item_id, ImrItem.tenant
if not item:
    raise HTTPException(status_code=404, detail="Seda rakenduskava meedet

new_status = update_data.get("status")
if new_status == "R":
    details = update_data.get("implementation_details", item.implementati
    if not details or len(details.strip()) < 15:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Tõrge: Meedet ei saa märkida rakendatuks (R) ilma pii
        )

    # Polymorphic verification check against evidence_links
    has_proof = db.query(EvidenceLink).filter(
        EvidenceLink.target_id == item.id,
        EvidenceLink.target_type == "imr_item",
        EvidenceLink.tenant_id == tenant_id
    ).count() > 0

    if not has_proof:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Tõrge: Rakendatud (R) meede nõuab vähemalt ühe digita
        )

# Apply updates
for key, value in update_data.items():
    if hasattr(item, key):
        setattr(item, key, value)

item.updated_by = user_id
item.updated_at = datetime.datetime.utcnow()
db.commit()

return {
    "message": "Rakenduskava meede edukalt uuendatud.",
    "imr_item_id": item.id,
    "status": item.status
}

}

@staticmethod
def update_process_protection_need(
    db: Session,
    tenant_id: uuid.UUID,
    process_id: uuid.UUID,
    new_levels: Dict[str, str]
) -> Dict[str, Any]:
    """
    E-ITS Business Rule: Once a tenant has chosen and saved a Mode of Protect
    the protection need level for any Business Process becomes strictly immut
    """

    tenant = db.query(Tenant).filter(Tenant.id == tenant_id).first()

    # Lock check
    if tenant and tenant.mode_of_protection is not None:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Kaitsetarvet ei saa muuta, sest turbeviis (põhi-, standar
        )

```

```

        protectionneedsSummary.business_process_id == process_id,
        ProtectionNeedSummary.tenant_id == tenant_id
    ).first()

    if not summary:
        raise HTTPException(status_code=404, detail="Kaitsetarbe kokkuvõtet e

    summary.confidentiality_need = new_levels.get("confidentiality", summary.
    summary.integrity_need = new_levels.get("integrity", summary.integrity_ne
    summary.availability_need = new_levels.get("availability", summary.availa
    db.commit()

    return {"message": "Äriprotsessi kaitsetarve edukalt uuendatud."}

```

4. API v2 Endpoint Definitions (backend/app/api/v2/endpoints/imr.py)

All API routes utilize strict Dependency Injection for database sessions and verified authorization context blocks.

```

from fastapi import APIRouter, Depends, Query, status
from sqlalchemy.orm import Session
import uuid
# Import your verified security dependencies: get_db, get_current_active_user

router = APIRouter(prefix="/imr", tags=["E-ITS Rakenduskava (IMR)"])

@router.patch("/items/{item_id}", status_code=200)
def update_imr_measure(
    item_id: uuid.UUID,
    status: str = None,
    responsible_id: uuid.UUID = None,
    due_date: str = None,
    implementation_details: str = None,
    verification_method: str = None,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_active_user)
):
    """
    Updates the operational details, responsibility and status of an active IMR m
    """
    update_payload = {}
    if status: update_payload["status"] = status
    if responsible_id: update_payload["responsible_id"] = responsible_id
    if due_date: update_payload["due_date"] = due_date
    if implementation_details: update_payload["implementation_details"] = impleme
    if verification_method: update_payload["verification_method"] = verification_

    return ImrService.update_imr_item(
        db=db,
        tenant_id=current_user.tenant_id,
        user_id=current_user.id,
        imr_item_id=item_id,
        update_data=update_payload
    )

@router.patch("/business-process/{process_id}/protection-need", status_code=200)
def change_process_needs(

```

```

        confidentiality: str,
        integrity: str,
        availability: str,
        db: Session = Depends(get_db),
        current_user = Depends(get_current_active_user)
    ):
        """
        Saves new protection levels (C-I-A) for a process. Rejects with HTTP 400 if M
        """
        return ImrService.update_process_protection_need(
            db=db,
            tenant_id=current_user.tenant_id,
            process_id=process_id,
            new_levels={
                "confidentiality": confidentiality,
                "integrity": integrity,
                "availability": availability
            }
        )

```

5. Integration Test Specifications (Pytest)

These integration test pipelines must verify tenant database isolation and enforce E-ITS specific state locks.

```

# test_imr_v2_service.py
import pytest
import uuid
from sqlalchemy.orm import Session
from fastapi import HTTPException
from app.models import ImrItem, Tenant, ProtectionNeedSummary, BusinessProcess

def test_imr_implemented_status_rejection_without_evidence(db_session: Session):
    """
    Ensure status transition to Implemented (R) fails without linked S3 evidences
    """
    tenant_id = uuid.uuid4()
    user_id = uuid.uuid4()
    imr_item_id = uuid.uuid4()

    item = ImrItem(
        id=imr_item_id,
        tenant_id=tenant_id,
        measure_id=uuid.uuid4(),
        status="P",
        implementation_details=""
    )
    db_session.add(item)
    db_session.commit()

    # Attempt transition without details and evidence links
    with pytest.raises(HTTPException) as exc_info:
        ImrService.update_imr_item(
            db=db_session,
            tenant_id=tenant_id,
            user_id=user_id,
            imr_item_id=imr_item_id,

```



```

    )

    assert exc_info.value.status_code == 400
    assert "piisava teostuskirjelduseta" in exc_info.value.detail

def test_lockout_prevents_protection_need_modification(db_session: Session):
    """
    Verify that Business Process protection needs are locked if Tenant.mode_of_protection is STANDARD
    """
    tenant_id = uuid.uuid4()
    process_id = uuid.uuid4()

    # Lock tenant by choosing STANDARD mode of protection
    tenant = Tenant(id=tenant_id, mode_of_protection="STANDARD")
    process = BusinessProcess(id=process_id, tenant_id=tenant_id, name="Test")
    summary = ProtectionNeedSummary(
        business_process_id=process_id,
        tenant_id=tenant_id,
        confidentiality_need="NORMAL",
        integrity_need="NORMAL",
        availability_need="NORMAL"
    )

    db_session.add_all([tenant, process, summary])
    db_session.commit()

    # Expecting failure on change attempts due to active lock
    with pytest.raises(HTTPException) as exc_info:
        ImrService.update_process_protection_need(
            db=db_session,
            tenant_id=tenant_id,
            process_id=process_id,
            new_levels={"confidentiality": "HIGH", "integrity": "HIGH", "availability": "HIGH"}
        )

    assert exc_info.value.status_code == 400
    assert "Kaitsetarvet ei saa muuta" in exc_info.value.detail

```

6. Front-End Component Mockup (React & Tailwind CSS)

Use this complete component layout inside the client view to represent interactive IMR execution states, S3 file uploading simulated pathways, and protection lockout warnings.

```

import React, { useState } from 'react';

interface ImrItem {
  id: string;
  code: string;
  name: string;
  status: 'P' | 'A' | 'E' | 'R' | 'O';
  responsibleName: string;
  dueDate: string;
  details: string;
  hasEvidence: boolean;
}

```

```

    id: string;
    name: string;
    confidentiality: string;
    integrity: string;
    availability: string;
  }

export default function App() {
  // Configurable state simulating backend response variables
  const [modeOfProtection, setModeOfProtection] = useState<string | null>('STANDA

  const [processes, setProcesses] = useState<BusinessProcess[]>([
    { id: 'bp-1', name: 'Üldarstiabi osutamine (Põhitegevus)', confidentiality: 'I
    { id: 'bp-2', name: 'Palgaarvestuse teostamine (Tugitegevus)', confidentiality

  ]]);

  const [imrItems, setImrItems] = useState<ImrItem[]>([
    { id: '1', code: 'ISMS.1.M1', name: 'Turbealduse üldvastutuse võtmine', stat
    { id: '2', code: 'CON.3.M5', name: 'Regulaarne andmevarundus', status: 'E', r
    { id: '3', code: 'SYS.1.1.M2', name: 'Süsteemide uuendamise reeglid', status:

  ]]);

  const [selectedItem, setSelectedItem] = useState<ImrItem | null>(null);
  const [responsibleInput, setResponsibleInput] = useState('');
  const [dueDateInput, setDueDateInput] = useState('');
  const [detailsInput, setDetailsInput] = useState('');
  const [statusInput, setStatusInput] = useState<'P' | 'A' | 'E' | 'R' | 'O'>('P'
  const [evidenceAttached, setEvidenceAttached] = useState(false);
  const [notification, setNotification] = useState<{ msg: string; type: 'success'

  const triggerNotification = (msg: string, type: 'success' | 'error') => {
    setNotification({ msg, type });
    setTimeout(() => setNotification(null), 5500);
  };

  const handleEditItem = (item: ImrItem) => {
    setSelectedItem(item);
    setResponsibleInput(item.responsibleName);
    setDueDateInput(item.dueDate);
    setDetailsInput(item.details);
    setStatusInput(item.status);
    setEvidenceAttached(item.hasEvidence);
  };

  const handleSaveItem = () => {
    if (!selectedItem) return;

    // Enforce PEARO Implemented status validations
    if (statusInput === 'R') {
      if (detailsInput.trim().length < 15) {
        triggerNotification('Tõrge: Rakendatud (R) staatus nõuab kirjeldust (min
        return;
      }
    }
    if (!evidenceAttached) {
      triggerNotification('Tõrge: Rakendatud (R) staatus nõuab seotud asitõendi
      return;
    }
  }

  const updated = imrItems.map(item => {
    if (item.id === selectedItem.id) {

```

```

        ...item,
        responsibleName: responsibleInput,
        dueDate: dueDateInput,
        details: detailsInput,
        status: statusInput,
        hasEvidence: evidenceAttached
    };
}
return item;
});

setImrItems(updated);
setSelectedItem(null);
triggerNotification('Meetme andmed salvestatud ja seosed kontrollitud.', 'suc
});

const handleTryEditProtection = (processName: string) => {
    if (modeOfProtection) {
        triggerNotification(
            `LUKUSTATUD: Äriprotsessi '${processName}' kaitsetarvet ei saa muuta, ses
            'error'
        );
    } else {
        triggerNotification(`Kaitsetarbe muutmise aken avatud protsessile: ${proces
    }
};

return (
    <div className="min-h-screen bg-slate-50 py-8 px-4 sm:px-6 lg:px-8 font-sans"
        <div className="max-w-4xl mx-auto">

            {/* Main Dashboard Header */}
            <div className="bg-white rounded-xl shadow-sm border border-slate-200 p-6
                <div className="flex flex-col sm:flex-row justify-between items-start s
                    <div>
                        <h1 className="text-2xl font-bold text-slate-900">Infoturbe meetmet
                        <p className="text-xs text-slate-500 mt-1">Skoobi modelleerimisel l
                    </div>
                    <div className="bg-slate-50 border border-slate-200 rounded-lg p-3 te
                        <span className="text-xxs font-bold text-slate-400 uppercase tracki
                        <span className={`text-xs font-bold px-2 py-0.5 rounded inline-bloc
                            modeOfProtection ? 'bg-indigo-100 text-indigo-800' : 'bg-amber-10
                        `}>
                            {modeOfProtection ? `LUKUSTATUD: ${modeOfProtection}` : 'Määramat
                        </span>
                    </div>
                </div>
            </div>

            {/* Global Notifications Panel */}
            {notification && (
                <div className={`p-4 rounded-lg mb-6 border text-sm font-semibold trans
                    notification.type === 'success'
                        ? 'bg-emerald-50 text-emerald-800 border-emerald-200'
                        : 'bg-rose-50 text-rose-800 border-rose-200'
                `}>
                    {notification.msg}
                </div>
            )}

            {/* Business Process Lockout Monitoring Grid */}

```

```

<div className="flex justify-between items-center md-4">
  <h2 className="text-md font-bold text-slate-800">Äriprotsesside kaits
  {modeOfProtection} && (
    <span className="text-xs text-rose-600 font-bold flex items-center">
      🛡️ Lukustatud
    </span>
  )}
</div>

<div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
  {processes.map(p => (
    <div key={p.id} className="p-4 rounded-lg bg-slate-50 border border
    <div>
      <h3 className="font-bold text-slate-900 text-sm">{p.name}</h3>
      <div className="mt-2 flex gap-1.5">
        <span className="text-xxs font-semibold bg-white border px-1.
        <span className="text-xxs font-semibold bg-white border px-1.
        <span className="text-xxs font-semibold bg-white border px-1.
      </div>
    </div>
    <button
      onClick={() => handleTryEditProtection(p.name)}
      className={`mt-4 w-full text-center text-xs font-bold py-1.5 ro
      modeOfProtection
        ? 'bg-slate-200 text-slate-400 cursor-not-allowed'
        : 'bg-indigo-50 text-indigo-700 hover:bg-indigo-100'
      }`}
    >
      {modeOfProtection ? '🔒 Muutmine lukustatud' : 'Muuda kaitsetar
    </button>
  </div>
  )}}
</div>
</div>

{/* IMR Table View */}
<div className="bg-white rounded-xl shadow-sm border border-slate-200 ove
  <div className="px-6 py-4 border-b border-slate-200 bg-slate-50/50">
    <h3 className="text-md font-bold text-slate-800">Rakenduskava aktiivs
  </div>
  <table className="w-full text-left border-collapse text-sm">
    <thead className="bg-slate-50/75 border-b border-slate-200 text-xxs f
    <tr>
      <th className="p-4">Kood</th>
      <th className="p-4">Meede</th>
      <th className="p-4">Olek</th>
      <th className="p-4">Vastutaja</th>
      <th className="p-4">Tähtaeg</th>
      <th className="p-4 text-right">Tegevus</th>
    </tr>
  </thead>
  <tbody className="divide-y divide-slate-150">
    {imrItems.map(item => (
      <tr key={item.id} className="hover:bg-slate-50/50 transition-colo
        <td className="p-4 font-bold text-blue-800">{item.code}</td>
        <td className="p-4 max-w-xs truncate font-medium text-slate-900
        <td className="p-4">
          <span className={`px-2 py-0.5 rounded-full text-xxs font-bold
            item.status === 'R' ? 'bg-emerald-100 text-emerald-800' :
            item.status === 'E' ? 'bg-amber-100 text-amber-800' : 'bg-s
          }`}>

```

```

        </span>
      </td>
      <td className="p-4 text-xs text-slate-600">{item.responsibleNam
      <td className="p-4 text-xs text-slate-600">{item.dueDate || 'Mä
      <td className="p-4 text-right">
        <button
          onClick={() => handleEditItem(item)}
          className="text-xs text-indigo-600 hover:text-indigo-950 fo
        >
          Taida meedet
        </button>
      </td>
    </tr>
  </tbody>
</table>
</div>

```

```

{ /* Selected Measure Detail Editing Area */ }
{selectedItem && (
  <div className="bg-white p-6 rounded-xl shadow-sm border border-slate-2
    <div className="flex justify-between items-start mb-4 border-b border
      <div>
        <h2 className="text-lg font-bold text-slate-900">
          Meetme andmete täitmine: <span className="text-blue-800">{selec
        </h2>
        <p className="text-xs text-slate-500 mt-0.5">{selectedItem.name}<
      </div>
      <button onClick={() => setSelectedItem(null)} className="text-slate
    </div>

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-4">
      <div>
        <label className="block text-xxs font-bold text-slate-500 upperca
        <input
          type="text"
          className="w-full border border-slate-200 rounded-lg p-2.5 text
          value={responsibleInput}
          onChange={(e) => setResponsibleInput(e.target.value)}
          placeholder="nt Juhatusesimees"
        />
      </div>
      <div>
        <label className="block text-xxs font-bold text-slate-500 upperca
        <input
          type="date"
          className="w-full border border-slate-200 rounded-lg p-2.5 text
          value={dueDateInput}
          onChange={(e) => setDueDateInput(e.target.value)}
        />
      </div>
    </div>

    <div className="mb-4">
      <label className="block text-xxs font-bold text-slate-500 uppercase
      <select
        className="w-full border border-slate-200 rounded-lg p-2.5 text-s
        value={statusInput}
        onChange={(e) => setStatusInput(e.target.value as any)}
      >
        <option value="P">P - Kavandatud (Planned)</option>

```

```

      <option value="E">E - Rakendamisel (In Progress)</option>
      <option value="R">R - Rakendatud (Implemented)</option>
      <option value="O">O - Väljast tellitud (Outsourced)</option>
    </select>
  </div>

  <div className="mb-4">
    <label className="block text-xs font-bold text-slate-500 uppercase">
      <textarea
        className="w-full border border-slate-200 rounded-lg p-2.5 text-slate-500"
        value={detailsInput}
        onChange={(e) => setDetailsInput(e.target.value)}
        placeholder="Kirjelda tegevused, mis tõendavad meetme toimimist.."
      />
    </div>

    <div className="mb-6 p-3.5 rounded-lg bg-slate-50 border border-slate-200">
      <input
        type="checkbox"
        id="evidence_check"
        checked={evidenceAttached}
        onChange={(e) => setEvidenceAttached(e.target.checked)}
        className="rounded border-slate-300 text-indigo-600 focus:ring-indigo-600"
      />
      <label htmlFor="evidence_check" className="text-xs font-semibold text-slate-500">
        📎 Digitaalne asitõend on seostatud (Faili info asub S3-s, link a
      </label>
    </div>

    <div className="flex justify-end gap-3 border-t border-slate-100 pt-4">
      <button
        onClick={() => setSelectedItem(null)}
        className="bg-slate-100 hover:bg-slate-200 text-slate-700 font-bold"
      >
        Tühista
      </button>
      <button
        onClick={handleSaveItem}
        className="bg-indigo-600 hover:bg-indigo-700 text-white font-bold"
      >
        Salvesta meetme andmed
      </button>
    </div>
  </div>
)}
</div>
</div>
);
}

```

7. Operational Implementation Checklist for the Coding Agent

Follow this systematic step-by-step pathway to deploy the v2 IMR and Modeling components safely:

Step 1: Execute SQL Migration

matches the status on the target environment.

Step 2: Implement Validations on `ProtectionNeedSummary`

In the service class managing protection needs, intercept update payloads and assert that the related `Tenant` does not have a defined `mode_of_protection` value yet. Raise `HTTPException 400` with a localized error message (*Törge...*) if locked.

Step 3: Write Polymorphic Evidence Linking Controller

Configure endpoint `POST /api/v2/evidences/link` to accept `target_id` (representing the `imr_item_id`) and `target_type` (`'imr_item'`). Validate S3 file metadata parameters before creating the database row.

Step 4: Write Baseline modeling generation tasks